C++ONLINE

ZUBIN NIRO SINGH

OPEN CONTENT:

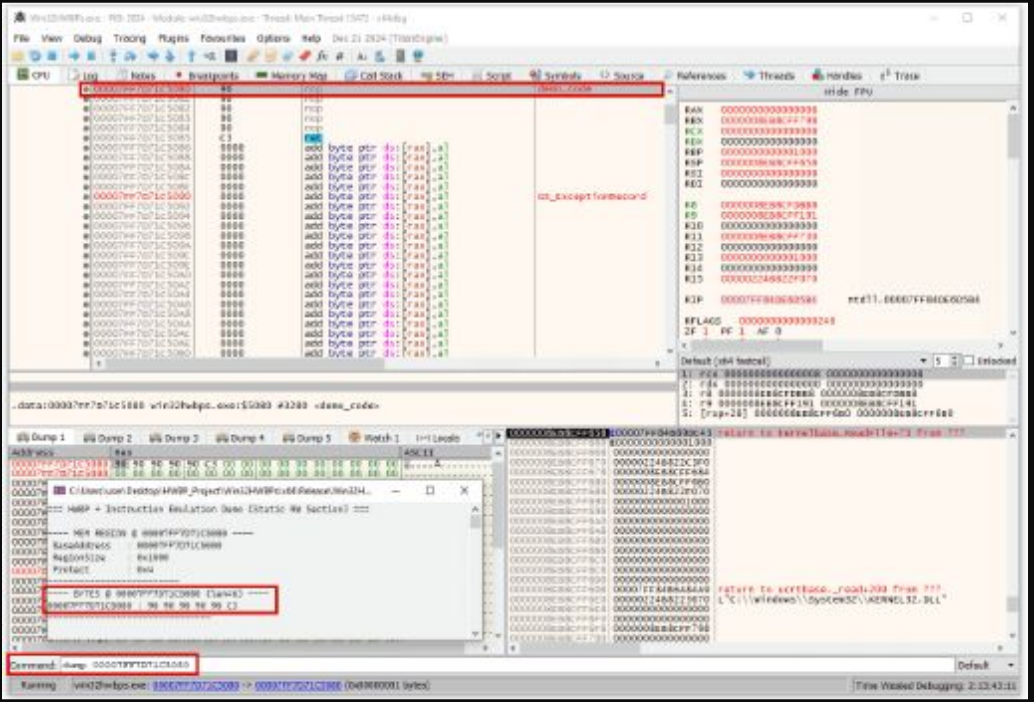
INSIDE THE MIND OF AN EXPLOIT

C++ TECHNIQUES FOR MALWARE DEVELOPMENT

2026

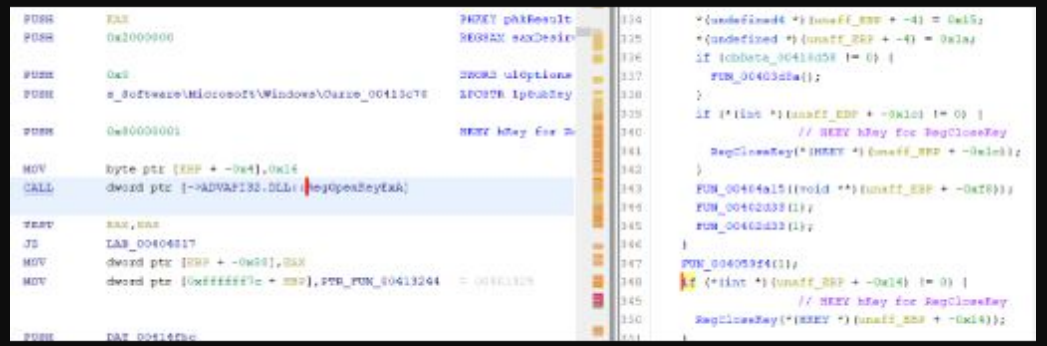
WHAT IS SHELLCODE?

- **The Definition:** Shellcode is a tiny piece of code (often written in Assembly or C) that is compiled down into raw, position-independent machine instructions.
- **The Goal:** Usually to spawn a command shell, execute a payload, or open a network connection back to the attacker.
- **The Delivery:** Attackers inject these raw bytes into the memory of a running process (like a web browser) and trick the CPU into executing them.



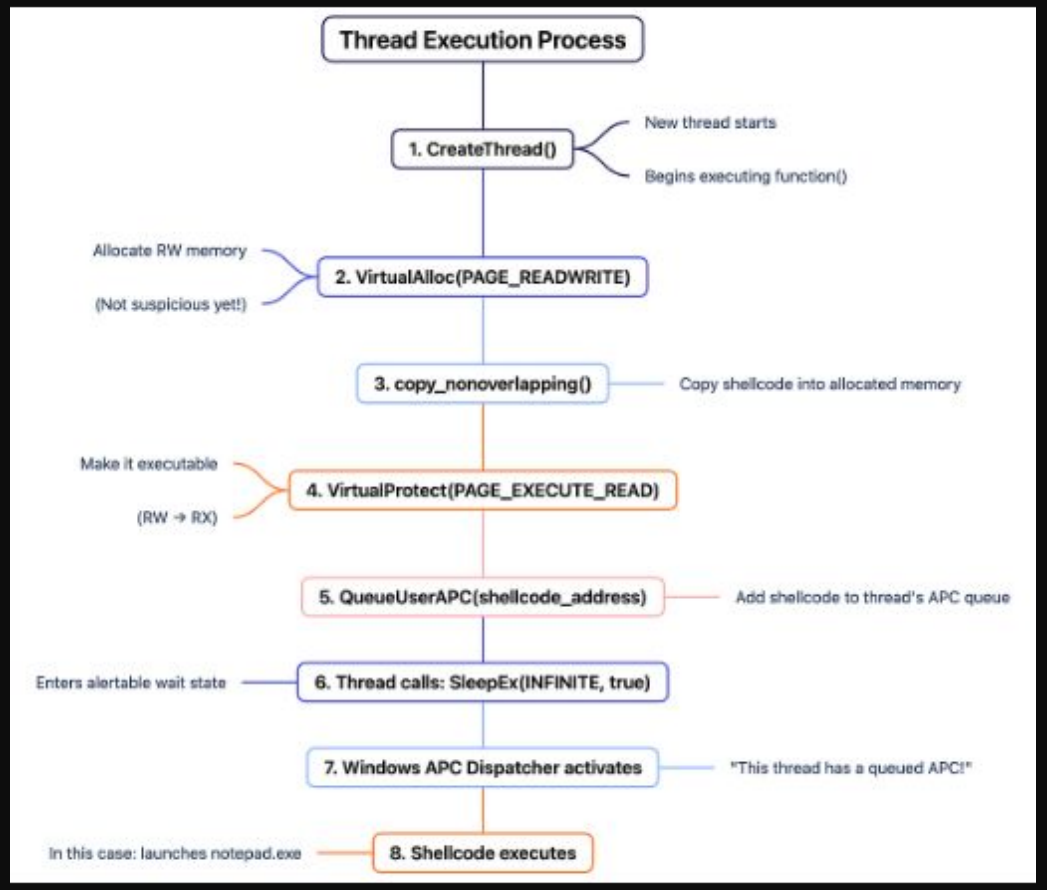
THE THREAT LANDSCAPE & TOOLS

- **Why are we here?** To understand how attackers bypass modern defenses, we must build the evasion techniques ourselves.
- **The Offensive Arsenal:** Payloads generated via msfvenom or Cobalt Strike.
- **The Defensive Mindset:** EDRs look for specific patterns. Our goal is to use the *C++ Compiler* as our primary weapon to break those patterns natively.
- **Tools used today:** Visual Studio (MSVC), C++20/23/26, and decompilers (IDA/Ghidra) to verify the obfuscation.



THE MALWARE TELL

- **Predictable Static Artifacts:** Hardcoded APIs, plaintext strings (e.g., ntdll.dll).
- **Predictable Behavioral Artifacts:** Allocate → Copy → Protect → Execute.
- **Predictable Analysis:** A straight, obvious line in the Call Graph that heuristics easily flag.



COMPILE-TIME STRING OBFUSCATION

Standard C

```
const char url[] =
    "http://c2.com";
```

The plaintext string is embedded directly in the binary, effortlessly extracted by basic forensic tools like strings.exe.

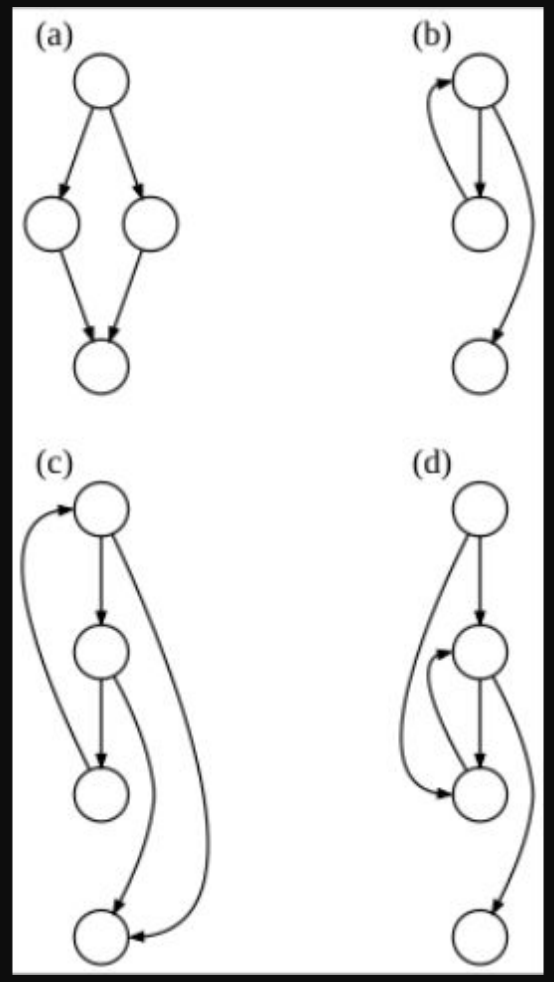
Modern C++20

```
constexpr auto url =
  ConstevalObfuscator("http://c2.com");
```

- consteval forces compile-time execution.
- Plaintext string **never** touches the .rdata section.
- **Demo Cue:** strings.exe comparison.

POLYMORPHIC CODE GENERATION

- **Compiler as an Engine:** Use template metaprogramming recursively inserting dummy math loops.
- **Unique Binaries:** Use macros like __TIME__ and __COUNTER__ to alter generation.
- **Drastic Shifts:** Every single build produces a completely unique binary hash and a different Control Flow Graph (CFG).
- **Zero Dependencies:** No external obfuscators needed; it's all native.



DYNAMIC EVASION VIA C++ FEATURES

Dynamic Polymorphism

Hide execution strategies behind virtual tables (indirect call [rax+N]), confusing static flow analysis.

Type Erasure

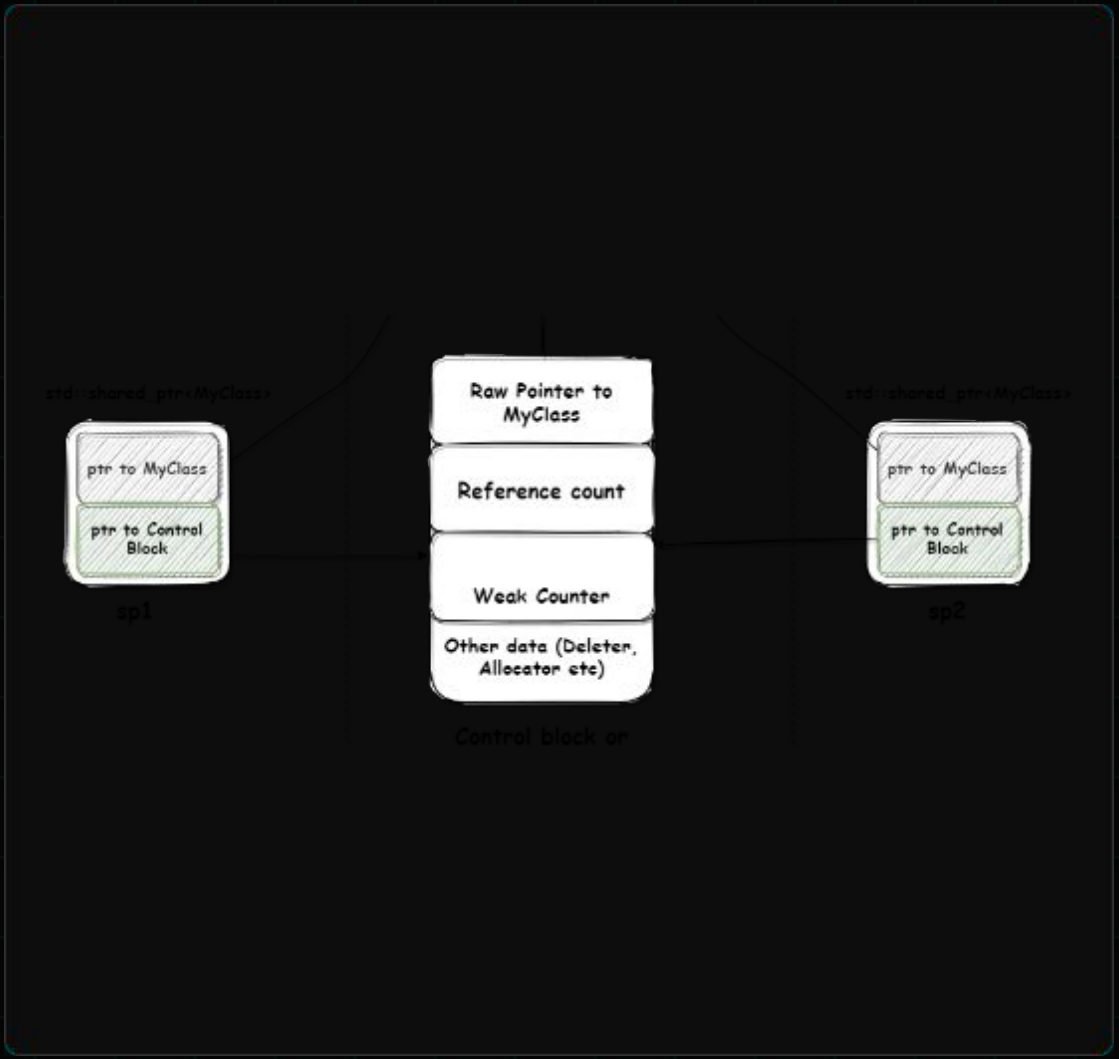
Pack shellcode pointer into a std::any object, effectively severing static data links from the payload.

Exception Flow

Throw the type-erased context as an exception. Actual execution happens inside a disconnected catch handler.

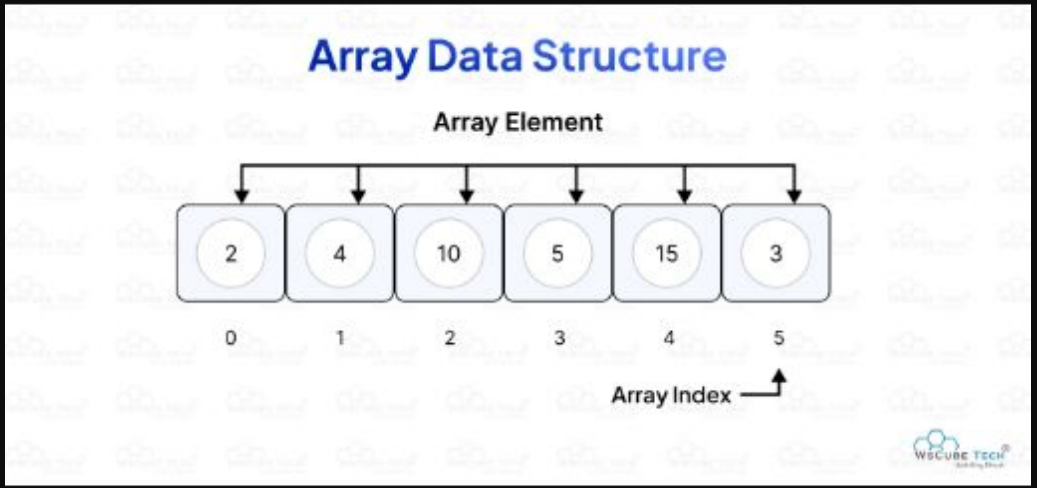
RAII DETONATION & MONADIC FLOW

- **RAII:** Execute shellcode inside the custom deleter of a std::unique_ptr.
- **Implicit Execution:** The payload detonates *implicitly* during stack unwinding.
- **Monadic Flow:** Use std::expected to chain staging steps with .and_then().
- **Result:** Produces a monolithic Control Flow Graph with zero branches and no explicit execute() call.



MULTI-DIMENSIONAL RECONSTRUCTION

- Stop encrypting shellcode. **Change its structure.**
- **consteval:** Scramble the 276-byte payload into a 6x46 2D matrix during compilation.
- **std::mdspan:** Zero-cost multi-dimensional view over the scrambled .rdata.
- **std::generator:** Lazily yield the correct sequential bytes into executable memory.



BREAKING STACKS WITH COROUTINES

Standard Thread

```
ntdll
  -> BaseThreadInit
    -> [Unbacked Memory]
```

Traditional execution threads have highly predictable stacks that EDRs monitor for unbacked memory anomalies.

C++20 Coroutine

```
main
  -> resume
    -> await_suspend
      -> [Payload]
```

- Coroutines compile into heap-allocated state machines.
- Shellcode execution fires from within heavily fragmented standard library infrastructure.

THE COST OF ABSTRACTION

- **The Tradeoff:** Weaponizing "Zero-Cost Abstractions" impacts final binary size on disk.
- Statically linking the C++ Standard Library and generating huge template instantiations inflates the binary.
- Larger sizes sometimes evade size-restricted scanners, but stick out conceptually as "bloated" tooling.

THE BLUE TEAM PERSPECTIVE

If static analysis and userland stack traces are blind, how do defenders win?

Kernel Telemetry

Rely on true kernel callbacks (ETWti / eBPF) for memory allocations and thread creations.

Memory Scanning

The payload must be decrypted eventually. Periodic YARA sweeps of the heap catch lazy-decrypted blobs.

Hardware Telemetry

CPU Performance Monitoring Counters (PMCs) can detect anomalies in execution behavior that software abstracts away.

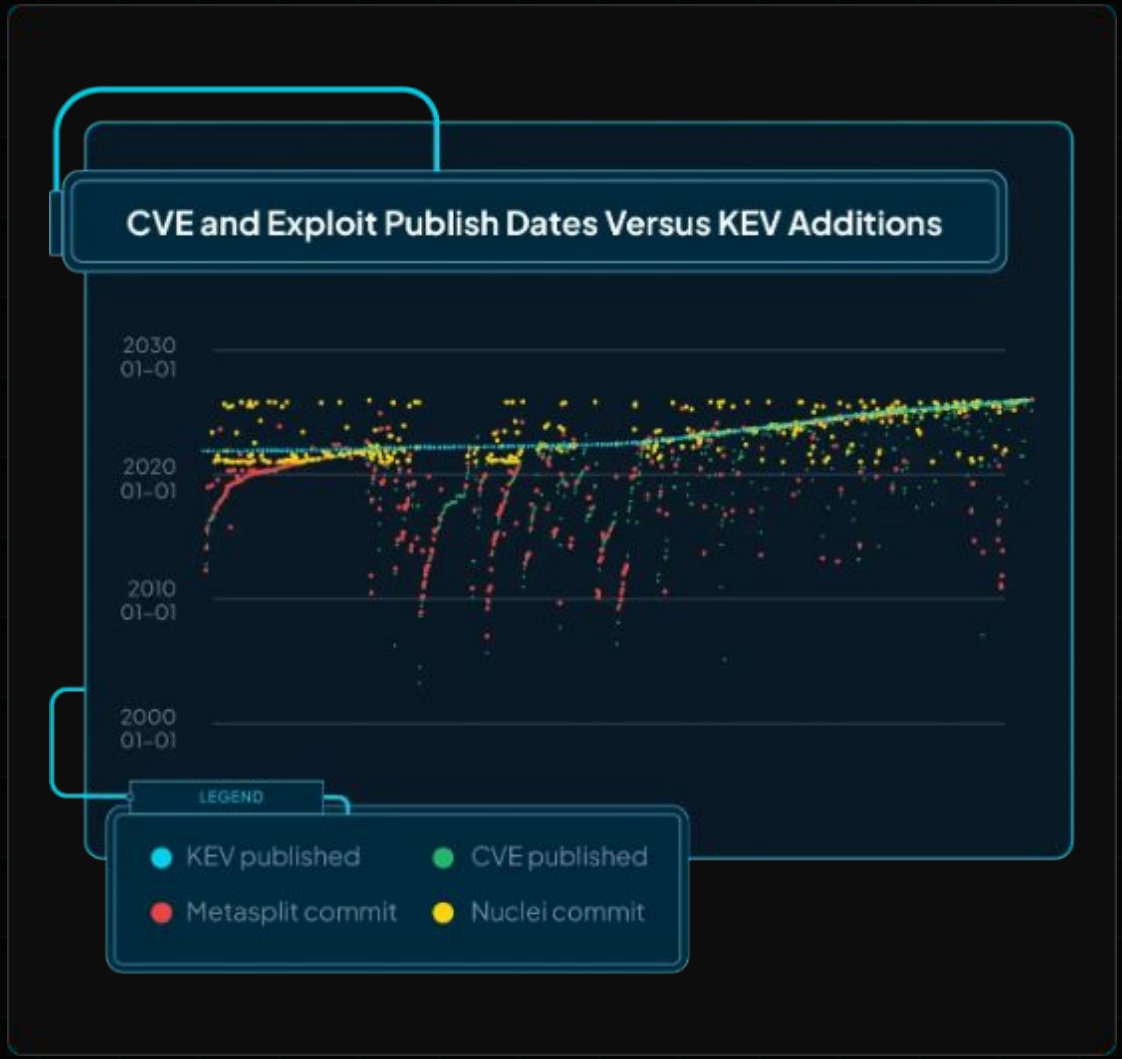
C++26 REALITY CHECK (REFLECTION)

- **Reflection:** std::meta::enumerators_of enables compile-time Syscall resolution directly to SSN.
- Generates raw mov r10, rcx; syscall blocks inline via consteval and asm volatile.
- **Reality:** C++26 is not fully ratified. Compiler support is experimental.

- This is *The Future of Evasion*, not an immediate off-the-shelf weapon today.

THE CALC POP MASTERCLASS

- **The Pipeline:** Compile-time encryption → Lazy decryption streaming → Coroutine heap execution.
- Utilizes 100% legitimate, standard C++ abstractions.
- Annihilates traditional heuristic analysis by looking exactly like modern enterprise architecture.
- It blends in by hiding within the sheer noise of the C++ standard library.



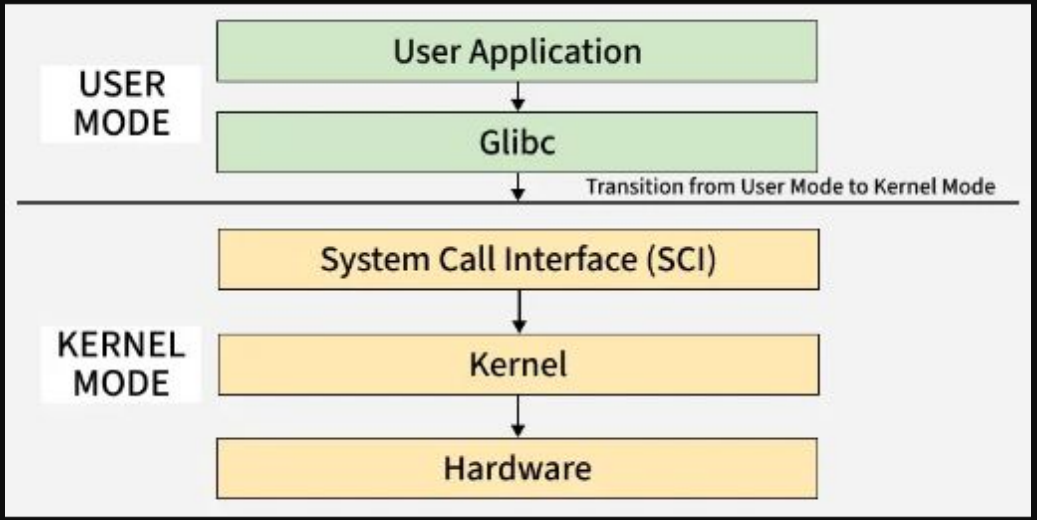
BEYOND THE SESSION: FUTURE WORK

> **What's Next?** We covered user-mode language abstractions. The journey doesn't stop here.

> **Deeper Dives:**

- Indirect Syscalls via Hardware Breakpoints (Exceptions).
- Bring Your Own Vulnerable Driver (BYOVD) for Kernel Pointers.
- Weaponizing WebAssembly (WASM) inside edge browsers.

> **Follow the Work:** Code for today will available at [https://github.com/zubsingh]. I'll be posting deeper research soon. You can get update on LinkedIn



Q&A

Questions ?

Connect Me on LinkedIn for more:

SCAN ME